



Pichulman Miguel Angel

TRABAJO PRACTICO N°7

INTEGRIDAD Y TRANSACCIONES

Parte 1: Fundamentos de la Integridad y Consistencia

- 1. Defina qué es la integridad de los datos y explique por qué es crucial en los sistemas de bases de datos. Mencione los tres tipos de restricciones de integridad descritos en el texto, dando un ejemplo para cada una.**

La integridad de los datos es el conjunto de procesos y reglas que garantizan que los datos almacenados en un sistema sean precisos, completos, consistentes y válidos durante todo su ciclo de vida. Es crucial porque permite que los datos sean confiables para la toma de decisiones, el análisis y la operación de las aplicaciones que dependen de ellos.

Tipos:

-Integridad de entidad: cada fila debe ser identificable de manera única. Se crea una tabla Alumnos y establece como PRIMARY KEY id_alumno, ningún registro puede tener el mismo valor y tampoco ser nulo.

-Integridad referencial: impide que existan referencias a registros que aun no existen en otra tabla. Se crea otra tabla llamada Pago, que registra los pagos de los alumnos, tiene un atributo id_alumno el cual se establece como FOREIGN KEY que hace referencia al atributo id_alumno en la tabla Alumnos

-Integridad de dominio: controla los valores permitidos por una columna (tipo de dato, restricciones). El atributo monto en la tabla Pago se define como un numérico positivo.

- 2. Utilizando el ejemplo de la transferencia bancaria provisto en el texto, explique el concepto de consistencia de la base de datos. ¿Cómo asegura la consistencia que el estado final de la base de datos sea válido después de la operación?**

La consistencia garantiza que toda transacción en la base de datos, transforme el sistema desde un estado válido a otro también válido según las reglas definidas.

Durante la operación, la suma total de dinero no cambia, es decir, el sistema impide que se cree dinero de la nada o desaparezca. Si ocurre un error, la base de datos NO consideraría la transacción como válida y revertiría cualquier cambio.

La consistencia garantiza que después de cualquier transacción, las reglas de integridad y negocio sigan cumplidas. Así, el estado final de la base de datos es siempre válido y coherente con la realidad.

- 3. ¿Qué se entiende por disponibilidad de la base de datos? Describa los aspectos clave que garantizan que una base de datos permanezca accesible y operativa.**

La disponibilidad se refiere a la capacidad de la base de datos para permanecer accesible y operativa incluso cuando múltiples usuarios la están accediendo simultáneamente.

Pichulman Miguel Angel

Aspectos clave:

Tiempo de actividad y confiabilidad del sistema: Mide cuánto tiempo la base de datos está funcionando sin interrupciones, asegurando que los datos estén accesibles para usuarios y aplicaciones cada vez que sea necesario.

Tiempo de respuesta bajo carga: Refleja la capacidad del sistema para ofrecer respuestas rápidas aun cuando muchos usuarios lo utilizan simultáneamente, evitando demoras y cuellos de botella.

Capacidad para manejar acceso concurrente: Permite que varios usuarios realicen operaciones a la vez sin causar conflictos ni pérdidas de datos, mediante la gestión eficiente de bloqueos y transacciones

Recuperación de fallas: Incluye mecanismos como respaldos, replicación y clústeres que posibilitan restaurar el servicio rápidamente tras un error, manteniendo la integridad y minimizando la pérdida de información.

Parte 2: Transacciones y Propiedades ACID

1. ¿Qué es una transacción de base de datos? Enumere y describa sus características clave.

Una transacción en bases de datos es una unidad de trabajo compuesta por una o varias operaciones que se ejecutan de forma indivisible. Es decir, todas las operaciones de la transacción deben completarse correctamente para que los cambios se confirmen y sean permanentes; si alguna falla, se deshacen todos los cambios realizados, dejando la base de datos en su estado original.

Características clave:

- Se ejecuta como una sola unidad indivisible
- Todas las operaciones se completan con éxito (confirmación) o ninguna lo hace (reversión)
- Mantiene la consistencia de la base de datos
- Aísla las operaciones concurrentes

2. Las propiedades ACID son los "bloques de construcción" de operaciones de base de datos confiables.

Para cada una de las propiedades (Atomicidad, Consistencia, Aislamiento, Durabilidad):

Defina la propiedad y su propósito.

Explique cómo se logra o qué garantiza en el contexto de una transacción.

Proporcione un ejemplo claro de cómo su violación podría afectar la integridad de los datos.

Puede usar ejemplos del material o crear uno propio.

Atomicidad

Definición y propósito: Una transacción es atómica: se ejecuta completamente o no se ejecuta en absoluto. Su propósito es evitar estados intermedios o parciales en la base de datos.

Cómo se logra: El sistema registra los pasos de la transacción y, si ocurre un error, revierte todos los cambios realizados (rollback). Solo si todos los pasos se completan, se confirman los cambios (commit).



Pichulman Miguel Angel

Ejemplo de violación: Si al transferir dinero entre cuentas solo se descuenta el monto de la cuenta origen y no se suma a la cuenta destino por un fallo, la base de datos queda en un estado incorrecto: el dinero "desaparece".

Consistencia

Definición y propósito: Garantiza que la base de datos pase de un estado válido a otro también válido, cumpliendo todas las reglas y restricciones definidas.

Cómo se logra: El sistema verifica que las operaciones respeten las reglas de integridad (por ejemplo, no permitir saldos negativos o duplicados en claves primarias). Si una transacción viola una regla, se rechaza.

Ejemplo de violación: Si una transacción permite que una cuenta tenga saldo negativo, se rompe la regla de negocio y la base de datos queda inconsistente.

Aislamiento

Definición y propósito: Cada transacción debe ejecutarse como si fuera la única en el sistema, sin interferencias de otras transacciones concurrentes. Su propósito es evitar conflictos y resultados inesperados.

Cómo se logra: El sistema utiliza bloqueos y niveles de aislamiento para que los cambios de una transacción no sean visibles para otras hasta que se complete.

Ejemplo de violación: Si dos usuarios transfieren dinero desde la misma cuenta al mismo tiempo y ambas transacciones leen el mismo saldo inicial, podrían terminar retirando más dinero del que realmente hay, generando un saldo negativo o inconsistente.

Durabilidad

Definición y propósito: Una vez que una transacción se confirma, sus cambios son permanentes y sobreviven a fallos del sistema.

Cómo se logra: Los datos se guardan en almacenamiento persistente (disco, backups). Si hay un corte de energía o reinicio, los cambios ya confirmados no se pierden.

Ejemplo de violación: Si un usuario realiza un pedido en una tienda online y el sistema confirma el pago, pero luego ocurre un apagón y el pedido desaparece, se pierde información crítica y la base de datos no es confiable.

3. **Observe el ejemplo de transacción SQL dado en el texto. Explique la importancia de las sentencias COMENZAR (BEGIN), CONFIRMAR (COMMIT) y DESHACER (ROLLBACK) en el contexto de la atomicidad.**

COMENZAR (BEGIN) define el inicio de la transacción. A partir de este punto, todas las operaciones (actualizaciones, inserciones, borrados) quedan agrupadas como un solo bloque de trabajo.

CONFIRMAR (COMMIT) finaliza y guarda de forma permanente todos los cambios realizados desde el COMENZAR. Solo si todas las acciones fueron exitosas, la base de datos hace efectivos los cambios.



Pichulman Miguel Angel

DESHACER (ROLLBACK) revierte todos los pasos hechos desde el COMENZAR, en caso de que alguna parte falle. Ningún cambio parcial queda guardado: la base de datos regresa al estado anterior al inicio de la transacción.

Parte 3: Control de Concurrencia

1. ¿Qué es la concurrencia en bases de datos y cuáles son los desafíos principales que presenta en entornos multiusuario?

La concurrencia se refiere a múltiples usuarios o procesos que acceden y potencialmente modifican los mismos datos simultáneamente. Esto es esencial en entornos multiusuario donde varios clientes pueden consultar o modificar registros al mismo tiempo, aumentando así la eficiencia y el rendimiento del sistema.

Desafíos en entornos concurrentes:

- Mantener la integridad de los datos
- Evitar interferencias entre transacciones
- Equilibrar el aislamiento con el rendimiento
- Evitar bloqueos y problemas de vivacidad

2. Enumere y describa los cuatro problemas de concurrencia comunes (Lecturas sucias, Lecturas no repetibles, Lecturas fantasma, Actualizaciones perdidas).

Lecturas sucias

Ocurre cuando una transacción lee datos que han sido modificados por otra transacción, pero esa segunda transacción aún no se ha confirmado. Si la segunda transacción es revertida, la primera transacción termina trabajando con datos incorrectos que nunca debieron existir.

Lecturas no repetibles

Sucede cuando una transacción vuelve a leer un mismo dato y encuentra un valor distinto porque otra transacción lo modificó y confirmó entre ambas lecturas. No hay consistencia en los resultados dentro de la misma transacción.

Lecturas fantasma

Aparecen o desaparecen filas en los resultados de una consulta durante el curso de una transacción porque otra transacción insertó o eliminó registros que cumplen con el criterio de búsqueda.

Actualizaciones perdidas

Dos transacciones modifican el mismo dato casi al mismo tiempo, pero la última actualización sobrescribe la anterior, perdiendo la modificación previa y dejando el sistema en estado inconsistente.



Pichulman Miguel Angel

- 3. Los niveles de aislamiento son fundamentales para gestionar la concurrencia. Mencione los cuatro niveles de aislamiento y explique qué problemas de concurrencia previenen o permiten cada uno.**

- READ UNCOMMITTED
- READ COMMITTED
- REPEATABLE READ
- SERIALIZABLE

¿Cómo se relaciona el nivel de aislamiento con el rendimiento de la base de datos?

Mientras mayor sea el nivel de aislamiento, menos concurrencia permite el sistema (más bloqueos durante más tiempo), aumentando la integridad pero reduciendo la eficiencia y la velocidad de respuesta.

Los niveles más bajos permiten mayor acceso simultáneo y respuestas rápidas, pero pueden provocar problemas de consistencia.

La elección adecuada depende de las necesidades del negocio y del tipo de datos manejados.

Demuestre cómo se establece un nivel de aislamiento para una transacción en SQL.

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
  
BEGIN TRANSACTION;  
  
    -- operaciones SQL  
  
COMMIT;
```

También se puede usar READ COMMITTED, SERIALIZABLE o READ UNCOMMITTED según necesidad.

- 4. Describa los tres mecanismos de control de concurrencia mencionados en el texto (Bloqueo, Ordenamiento por marca de tiempo, Control de concurrencia de múltiples versiones). En el caso del bloqueo, explique los tipos de bloqueos y cómo previene problemas como las actualizaciones perdidas, haciendo referencia al ejemplo visual del usuario A y B.**

-Bloqueo: Restringe el acceso a los elementos de datos que están siendo utilizados por otras transacciones.

Ejemplo (usuario A y B):

Cuando el usuario A comienza a modificar el inventario, adquiere un bloqueo exclusivo sobre ese producto. Si el usuario B intenta modificar el mismo producto, debe esperar hasta que A termine y libere el bloqueo (al confirmar su transacción con "commit"). Así se previenen actualizaciones perdidas, ya que solo se permite una modificación a la vez, garantizando coherencia y evitando sobrescribir cambios de otros usuarios.

-Ordenamiento por marca de tiempo: asigna marcas de tiempo a las transacciones y ejecuta las operaciones en función del orden de las marcas de tiempo.

-Control de concurrencia de múltiples versiones: mantiene múltiples versiones de los elementos de datos para permitir operaciones de lectura sin bloqueo.



Pichulman Miguel Angel

4. ¿Qué es un bloqueo mutuo (deadlock) y cómo se resuelven?

Un bloqueo mutuo (o deadlock) es una situación en la que dos o más transacciones quedan permanentemente bloqueadas porque cada una está esperando que la otra libere un recurso necesario para continuar. Por ejemplo, la transacción A bloquea el recurso 1 y espera el recurso 2, mientras que la transacción B bloquea el recurso 2 y espera el recurso 1. Ninguna puede avanzar y el sistema entra en un estado de espera circular.

¿Cómo se resuelven los bloqueos mutuos?

Detección y eliminación: Los sistemas de bases de datos detectan automáticamente el deadlock vigilando los recursos bloqueados y las dependencias. Cuando se detecta, el gestor escoge una de las transacciones involucradas como "víctima", la aborta (rollback) y libera sus recursos para que las demás puedan continuar.

Prevención: Se aplican técnicas como requerir los recursos siempre en el mismo orden, evitando así ciclos de espera circular.

Evitación: El sistema analiza si una nueva solicitud de recurso puede llevar a deadlock antes de concederla.

Parte 4: Aplicaciones en el Mundo Real y Preguntas para Investigar

1. Identifique y explique tres aplicaciones del mundo real donde las transacciones son críticas para mantener la integridad y consistencia de los datos.

Sistemas Bancarios y Financieros (Transferencias, Cajeros Automáticos):

Una transferencia de fondos debe ser completamente atómica (ej. débito y crédito deben ocurrir juntos) y consistente (el balance total debe conservarse). Un fallo en la atomicidad podría resultar en la pérdida de dinero para el banco o el cliente.

Sistemas de Comercio Electrónico (E-commerce) y Gestión de Inventario:

Una transacción de compra debe ser atómica (debitar el pago y disminuir el inventario al mismo tiempo). El aislamiento es clave para evitar la doble venta del último artículo en stock (problema de actualización perdida). La durabilidad garantiza que una vez que el cliente paga, la compra es registrada permanentemente.

Sistemas de Reservas (Vuelos, Hoteles, Boletos):

Una transacción de reserva debe bloquear el asiento, procesar el pago y actualizar el inventario de disponibilidad. El aislamiento asegura que otra transacción no pueda reservar el mismo asiento mientras la primera está en curso.



Pichulman Miguel Angel

2. Preguntas de Investigación (requiere información fuera del texto proporcionadas):

Pregunta 1: Investigue y compare dos estrategias diferentes para manejar los problemas de concurrencia que no se hayan detallado exhaustivamente en el texto (por ejemplo, el control de concurrencia optimista versus pesimista, o diferentes tipos de bloqueos como bloqueos de intención). Explique cuándo sería apropiado usar una sobre la otra.

- Sitios web sugeridos para la investigación: Documentación oficial de sistemas de gestión de bases de datos (por ejemplo, PostgreSQL, MySQL, Microsoft SQL Server), blogs técnicos especializados en bases de datos (ej. Percona, Redgate, blogs de proveedores de nube como AWS o Google Cloud), artículos académicos o de conferencias sobre control de concurrencia.

1. Control de concurrencia pesimista

El control pesimista asume que los conflictos de acceso concurrente son frecuentes. Por eso, cuando una transacción quiere modificar un registro, lo bloquea (a nivel de fila, página o tabla), impidiendo que otro usuario lo lea o modifique hasta que la transacción termine (haga commit o rollback).

¿Cómo funciona?

Cuando un usuario adquiere un bloqueo sobre los datos, los demás usuarios quedan en espera hasta que el primero lo libere.

Garantiza que no ocurrirán actualizaciones perdidas o inconsistencias, a costa de menor concurrencia y posible aumento de esperas y bloqueos mutuos (deadlocks).

¿Cuándo es recomendable?

En entornos donde los conflictos de acceso simultáneo son comunes y los datos tienen alta contención (por ejemplo, procesamiento de transacciones bancarias, sistemas de inventario críticos).

Cuando las transacciones son cortas y frecuentes, para minimizar el tiempo de bloqueo y brindar alta seguridad.

2. Control de concurrencia optimista (OCC)

¿En qué consiste?

El OCC asume que los conflictos de acceso concurrente son raros. Por ello, las transacciones avanzan sin bloquear los datos. Antes de hacer commit, el sistema valida que los datos que la transacción leyó y quiere modificar no hayan sido cambiados por otra transacción. Si todo está bien, la transacción se confirma; si hubo un cambio, se deshace (rollback) y puede reintentarse.

Fases típicas del OCC:

Lectura: Lee y trabaja sobre una copia de los datos actual.

Validación: Antes de escribir, verifica que otros no hayan modificado los datos involucrados.

Pichulman Miguel Angel

Escritura: Si la validación es exitosa, los cambios se guardan; si no, se deshacen y la transacción puede reiniciarse.

¿Cuándo es recomendable?

En sistemas donde las transacciones suelen trabajar sobre datos distintos y los conflictos son poco frecuentes (por ejemplo, aplicaciones donde los usuarios principalmente consultan datos o actualizan registros independientes).

Es ideal para operaciones “desconectadas” o que involucren largas sesiones de usuario.

Pregunta 2: Elija un sistema de gestión de bases de datos (DBMS) específico (por ejemplo, PostgreSQL, MySQL, Oracle, SQL Server). Investigue cómo este DBMS implementa las propiedades ACID y cuáles son sus niveles de aislamiento predeterminados. ¿Existen diferencias significativas en la implementación de ACID o en la gestión de la concurrencia en comparación con lo que se describe de forma general en el módulo?

- Sitios web sugeridos para la investigación: Documentación oficial del DBMS elegido, foros de la comunidad de dicho DBMS, libros y cursos especializados en ese DBMS, blogs de ingenieros de bases de datos que trabajen con el sistema elegido.

Se ha elegido PostgreSQL como Sistema de Gestión de Bases de Datos (DBMS) ya que es un sistema robusto, open source y conocido por su estricto cumplimiento de los estándares de bases de datos.

1. Implementación de Propiedades ACID en PostgreSQL

PostgreSQL está diseñado para ser completamente ACID compliant.

Atomicidad (A) y Durabilidad (D): Se implementan a través del sistema de Write-Ahead Logging (WAL).

WAL: Asegura que todos los cambios en la transacción se escriban en un registro persistente (log) antes de que los cambios se apliquen realmente a los archivos de datos principales en el disco. Esto garantiza que si el sistema falla (ej., un corte de energía) antes de que el COMMIT llegue a los archivos de datos, la transacción aún pueda recuperarse completamente de los registros WAL en el reinicio del servidor, garantizando la Durabilidad. Si la transacción falla, el log permite el ROLLBACK para garantizar la Atomicidad.

Aislamiento (I): Se implementa principalmente mediante el Control de Concurrencia de Múltiples Versiones (MVCC).

MVCC: Permite que las operaciones de lectura accedan a versiones anteriores de las filas modificadas por otras transacciones que aún no han confirmado. Esto significa que las lecturas no bloquean las escrituras, y las escrituras no bloquean las lecturas, mejorando significativamente la concurrencia.

Consistencia (C): Se garantiza mediante la imposición de todas las restricciones de integridad (clave primaria, clave foránea, check, unicidad) y la reversión de transacciones que intenten violar estas reglas.

Pichulman Miguel Angel

2. Niveles de Aislamiento Predeterminados

El nivel de aislamiento predeterminado en PostgreSQL es READ COMMITTED.

3. Diferencias Significativas en la Gestión de la Concurrencia

Existe una diferencia clave entre la implementación de PostgreSQL (basada en MVCC) y la descripción teórica general (basada en bloqueos puros) en los manuales de bases de datos:

Característica	Descripción Teórica General (Estándar ANSI)	Implementación de PostgreSQL (MVCC)
Mecanismo Central	Principalmente Bloqueo (Locking).	Principalmente MVCC (Multi-Version Concurrency Control).
Rendimiento de Lectura	Las lecturas (bloqueos compartidos) a veces pueden ser bloqueadas por las escrituras (bloqueos exclusivos).	Las transacciones de lectura nunca bloquean las escrituras, ya que cada lector ve una instantánea de datos consistente. Esto proporciona un rendimiento superior en entornos con muchas lecturas.
Comportamiento en READ COMMITTED	Teóricamente, el nivel READ COMMITTED permite el problema de Lecturas No Repetibles.	Bajo MVCC, PostgreSQL evita el problema de Lecturas Sucias (como lo requiere el estándar) y proporciona una instantánea para cada sentencia, mitigando el impacto de Lecturas No Repetibles en comparación con otros sistemas, aunque el nivel que lo elimina por completo es REPEATABLE READ o SERIALIZABLE.